

张目远

Agent 开发 | RAG / Agent 系统 | 后端工程 | 19829773360 | charlizezmy@charlizezmy@gmail.com



教育背景

西北工业大学

软件工程·硕士（软件学院）

◦ GPA 3.77 / 4.1, 专业排名 top 10%

软件工程·学士（软件学院）

◦ GPA 3.54 / 4.1, 专业排名 top 15%, 保研至本校软件学院

985 / 211 · 陕西西安

2024.09 - 至今

2020.09 - 2024.06

实习经历

大疆 DJI 研发管理与AI使能部·绩效管理组

2026.07 - 至今

绩效评估 Agent 平台
(rd-points)

系统级共建者 | 全
栈/Agent 开发

Task Scheduling · Workflow Core · Claude SDK

◦ 项目背景：公司绩效管理系统由 Platform（SpringBoot 业务编排中枢）+ Sphere（Claude Agent 执行引擎）+ FE（React）三仓库协作，定时拉取公司各部门系统（飞书、JIRA、Colla 等）中的任务，调度给 Sphere 通过提示词使用各部门编写的 Skill 得到基础得分，任务完成后再次由 Sphere Review 完成情况并计算综合得分，并产出员工任务负荷、部门健康状况、贡献度排行、任务总池看板数据。

◦ 我的贡献：Sphere 评估结果 投递灰度升级、贡献度助手 SSE 流式推送（Agent 执行事件实时推送 + 断线续传）、任务调度吞吐治理落地。

◦ Webhook → Redis Stream 可靠投递：原方案 Sphere 终态经 HTTP webhook 回调 Platform，Platform 重启/发布期间回调重试耗尽后结果滞留 Sphere，需人工重投。改用 Redis Streams 消费组（较 Pub/Sub 多 PEL + 消息认领，适配消费者重启恢复）：Sphere 侧 Run 终态与投递 Outbox 同事务提交，Publisher 轮询以 XADD 写入 Stream（指数退避重试，超阈值转 DEAD_LETTER 保留人工重放）；Platform 侧以 XREADGROUP 消费、XAUTOCLAIM 认领超时 PEL 消息，靠终态幂等更新把 at-least-once 投递收敛为 effectively-once 生效，仅事务提交成功后 XACK。按 Run 灰度切换，开关经 Apollo 热更新驱动 AtomicBoolean CAS 无锁切换，保留 webhook 兼容路径。

◦ 贡献度助手 SSE 流式推送：Controller 建立 EventSource 长连接（SseEmitter，5 分钟强制到期、浏览器自动重连），经鉴权 + Redis INCR 限流（lease 租约）后转入线程池，以 XREAD BLOCK 阻塞轮询 Redis Stream 逐条转发；写入侧用 fencing token（Lua 原子脚本）保证仅当前活跃执行轮次可写事件，防止旧轮次覆盖；基于游标（Last-Event-ID 优先）实现断线续传，游标越界或早于故障恢复点时下发全量快照重建前端；15s 心跳兼做保活与权限热校验，Redis 故障统一降级为 stream.degraded。

◦ 任务调度吞吐治理：团队排查出调度链路瓶颈——批量提交在单线程上顺序执行、逐任务 6-10 次 N+1 DB 查询、结果回写 Stream 消费/Outbox 批次过小；我落地 Quick Win + Phase 1 两阶段修复：调优 poll-batch-size（10→50-100）/batch-size（100→300-500）/interval-ms（10min→2min）/Outbox fixedDelay（2s→500ms）等配置项，并改造代码消除 N+1（循环前批量预加载）、复用 sphereEvalExecutor 线程池并行化 Sphere 提交、新增超时自动失败兜底（定时扫描 + eval_status 物化）。上线后（2026-09-02~09-03 实测）30 分钟内完成率 23.27%→53.72%，2 小时内完成率 49.76%→85.81%，8 小时以上长尾占比 20.84%→1.66%，12 小时以上长尾清零。

AgentOps 沙箱管理
(rd-agent-be)

参与设计 + 部分实现
| Agent 代码执行沙
箱资产管理

DDD 六层架构 · 状态机 · 事件驱动 · OpenSandbox

- **项目背景：** 平台 Agent 执行代码生成/执行类任务需要隔离、可伸缩、可销毁的沙箱环境，此前 CPU/内存资源由后端硬编码分配，无法按需创建、调整或回收；引入阿里 **OpenSandbox** 作为底层运行时，在平台侧新增沙箱资产的可视化管理。
- **我的贡献：** 参与 sandbox 领域 **DDD** 六层架构（facade/client/domain/infra/application/adapter）设计与部分模块实现——聚合根设计为纯状态机（草稿→初始化→在线→下线/失败五态，容器创建/健康检查/销毁隔离在应用层，领域层不感知 **OpenSandbox**）；submit 同步落库并发领域事件，SandboxRunner 异步监听事件驱动容器供给（成功转 **online**，失败转 **FAILED**，靠状态前置校验兜底重复事件的幂等）；新增 SandboxReconcileScheduler 定时对账（**Redisson** 分布式锁防多副本并发），周期校正在线沙箱与 **OpenSandbox** 真实状态。

个人知识库 端到端链路 Owner | 从 0→1 构建 Plan-Verify-Replan 工作流与三层 Memory 架构 Agent

Python + LangGraph + FastAPI; 覆盖检索、验证、记忆、状态恢复与知识闭环

- 基于 LangGraph StateGraph 构建 Plan-Verify-Replan Agent, 设计 Planner / Retriever / Answerer / Verifier / Replanner / KnowledgeLoop 六节点工作流; Verifier 基于 RAGAS Faithfulness 校验答案忠实性, 未通过则触发重检索
- 设计 Hybrid Search (text-embedding-3-small + ChromaDB + BM25) + RRF 融合排序, 引入 Query Expansion; 构造 100+ 条 QA 评测集以 RAGAS 量化, Context Recall 相比 baseline 提升 36.6%, Faithfulness 达 0.843
- 设计三层 Memory 架构: LangGraph InMemoryStore 管理会话级短期记忆, Redis Hash 持久化用户长期阅读偏好与兴趣标签, PostgreSQL 存储历史问答记录; Memory 命中时跳过重复检索, 平均响应延迟从 602ms 降至 2ms
- 基于 LangGraph Checkpoint (SqliteSaver) 实现 Step-level 状态持久化, 检索或工具调用失败时自动回滚并触发降级策略 (扩大 top-k / 放宽过滤条件), 长链路任务故障恢复率 $\geq 95\%$
- 构建知识闭环机制: Agent 回答时若检测到笔记覆盖不足, 自动生成补充摘要写回知识库, 持续扩充个人知识图谱

基于 Spring Cloud 的电商交易系统

核心开发者

Spring Cloud 微服务架构; 覆盖库存、订单、检索与缓存链路

- 基于 Redis + Lua 实现库存原子扣减, 在秒杀压测场景下实现高并发请求的库存一致性控制与超卖防护
- 引入 RocketMQ 构建异步订单链路, 将下单流程异步化处理, 实现削峰填谷并降低数据库瞬时写入压力
- 采用 Redis + Lua + Redisson 构建分层并发控制机制, 在库存与非库存场景分别实现原子性扣减与防重复下单控制
- 采用 Canal + Binlog + MQ 构建最终一致性方案, 以数据库变更为事件源, 实现订单与库存的异步同步与状态修复
- 基于 Elasticsearch 构建商品多维检索能力, 支持复杂条件过滤与排序优化; 设计 Caffeine + Redis 二级缓存, 构建本地缓存优先的热点访问路径, 压测环境下显著降低数据库访问延迟

VideoSCI 视频压缩感知重建 (Differential Unfolding)

论文一作 (AAAI 2027 投稿中)

迭代框架重构与差分 Transformer 特征更新; 覆盖算法、训练与评测

- 针对 VideoSCI 重建中计算开销大、迭代后期特征更新停滞问题 (导致资源浪费与性能受限), 重构迭代框架, 解耦稳定结构与动态更新, 减少无效计算
- 基于差分 Transformer 优化特征更新方式, 提升动态建模能力
- 结果: PSNR 提升超过 0.5 dB, 参数量减少 50% 以上, 计算量减少 33% 以上, 实现更优的精度-效率权衡
- 论文状态: AAAI 2027 投稿中; 代码: github.com/Muyuan-Zhang/DU

专业技能

- 后端开发: Java / Python, 面向对象设计与常用集合框架, 并发编程基础 (JUC、线程池、锁机制)
- Web 框架: Spring Boot / Spring MVC / MyBatis / FastAPI, 理解 IoC、AOP 原理及常见设计模式
- 中间件: MySQL / Redis / MQ / Elasticsearch / ChromaDB 等, 缓存设计、消息队列削峰、分布式一致性
- 大模型应用: RAG / Agent 系统完整落地经验; LangGraph / LangChain, Plan-Verify-Replan 工作流、多层 Memory 设计与 Checkpoint 失败恢复